

5 模型诊断

实现机器学习算法的技巧。当我们训练模型或使用模型时，发现预测误差很大，可以考虑进行以下优化：

- (1) 获取更多的训练样本
- (2) 使用更少的特征
- (3) 获取其他特征
- (4) 添加多项式特征
- (5) 减小正则化参数 λ
- (6) 增大正则化参数 λ

如果对以上方法一一尝试或在错误的方向上进行调整会浪费大量的时间，下面介绍一些模型诊断方法来判断哪些因素对模型性能产生了影响，并确定优化方向。下面将介绍如何评估模型性能并做出正确的选择。

5.1 模型选择

对于一个任务，如何确定选择什么样的模型？例如，在线性回归问题中，想要构建一个多项式回归模型，如何确定多项式的阶数；在分类问题中使用神经网络，如何确定神经网络的层数和神经元个数。

为了确定一个模型是否合适，需要对模型进行评估。首先将数据集划分成训练集和测试集，划分比例通常为 7:3 或 8:2。当训练一个模型时，首先定义代价函数，代价函数可能包含正则化项，然后通过最小化训练集的代价来拟合参数。接下来在测试集上使用训练好的模型计算测试集的代价，这里要注意在测试集上计算代价时不包括正则化项，这一指标叫做**测试误差**。另一个衡量模型性能的指标是**训练误差**，它衡量算法在训练集上的指标，训练误差计算训练集的代价，同样不包括正则化项。

在模型选择过程中，首先定义好一些大概可行的模型，然后进行训练，在数据集上计算已经训练好的模型的误差，选择误差最小的模型作为最终模型。通常**训练误差**无法很好地衡量算法在新样本上的表现，而为了公平地评估模型性能，测试集要保留到最后用于模型的性能评估。因此，可以将数据集划分为训练集、交叉验证集（也称验证集）和测试集三部分，划分比例通常为 6:2:2。在交叉验证集上计算模型误差，并选择**交叉验证误差**最小的模型。以上就是模型选择的过程。

在参加比赛或者科研过程中通常需要对比不同模型的性能，使用交叉验证集进行模型选择，然后使用测试集进行评估，保证了评估结果的公平性，因为测试集中的数据是所有模型都没有见过的。

5.2 偏差（Bias）与方差（Variance）

当训练好的模型没有达到期望的效果时，接下来的关键在于决定下一步该做什么以提高模型性能。**偏差与方差**可以指导我们下一步应该尝试做什么。

前面在讲过拟合和欠拟合时，因为只有一个或两个特征，所以能够从图像上看出是否发生过拟合或欠拟合。然而，在实际问题中，通常有很多特征，无法绘制图像进行观察。因此，需要一个更系统的方式进行诊断，这里可以使用偏差与方差来诊断是否发生了过拟合或欠拟合。

偏差是指模型的预测值与真实值之间的差异，偏差越大，说明模型对数据的拟合能力较差，偏差越小，说明模型对数据的拟合能力较好。机器学习中偏差的计算方法如下：

$Bias = \text{预测值的平均值} - \text{真实值的平均值}$

方差是指模型在不同数据集上的预测结果的波动程度，反映了模型对数据变化的敏感度。方差衡量的是模型预测的稳定性，如果模型在不同数据集上预测偏差的差异很大，方差就大，反之，方差就小。方差的计算方法如下：

$$Variance = E[(f(x) - E[f(x)])^2]$$

其中 $f(x)$ 是模型在不同数据集上的预测值。当发生欠拟合时，模型在训练集和验证集上的表现都很差。此时训练偏差和验证偏差都比较高，方差低，模型过于简单，无法捕捉数据的复杂关系。当发生过拟合时，模型在训练集上的表现很好，但在验证集上的表现很差。此时训练偏差低验证偏差高，方差高，模型过于复杂，对训练数据的细节和噪声过于敏感。

在机器学习中，理想的情况是找到一个模型，其偏差和方差都较低，从而实现良好的泛化性能。然而，实际上，降低偏差往往会增加方差，反之亦然。这是因为增加模型的复杂性可以降低偏差，但同时也可能使模型对训练数据更加敏感，从而使方差增大。

Part5-1 动手练

绘制不同复杂度模型的偏差与方差

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# 加载数据
data = pd.read_csv('ex2data2.txt', names=['x1', 'x2',
'accepted'])

# 生成多项式特征
def map_feature(x1, x2, degree=5):
    out = np.ones((x1.shape[0], 1)) # 添加偏置项
    for i in range(1, degree+1):
        for j in range(0, i + 1):
            new_feature = (x1 ** (i - j)) * (x2 ** j)
            out = np.hstack((out, new_feature.reshape(-1, 1)))
    return out

# sigmoid函数
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# 正则化代价函数
def cost_reg(X, y, theta, lamda):
    m = len(y)
    epsilon = 1e-15 # 防止log(0)
    y_pred = sigmoid(np.dot(X, theta))
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    cost = -1 / m * (np.dot(y, np.log(y_pred)) + np.dot((1 - y),
np.log(1 - y_pred)))
    reg = (lamda / (2 * m) * np.sum(np.power(theta[1:], 2)))
    cost_reg = cost + reg
    return cost_reg

# 定义正则化梯度下降法
def gradientDescent(X, y, theta, learning_rate=0.01,
epochs=1000, lamda=1):
    m, n = X.shape
    cost = np.zeros(epochs) # 初始化代价函数值为0数组，元素个数为
迭代次数

    for i in range(epochs):
```

```

        y_pred = sigmoid(np.dot(X, theta))
        dw = (1 / m) * np.dot(X.T, (y_pred - y))
        reg = (lamda / m) * theta
        reg[0] = 0 # 不对偏置项进行正则化
        dw = dw + reg
        theta = theta - learning_rate * dw

    cost[i] = cost_reg(X, y, theta, lamda) # 计算每一次的代
价函数
    return theta, cost

# 使用训练好的模型进行预测
def predict_probabilities(theta, X):
    return sigmoid(np.dot(X, theta))

# 初始化参数
alpha = 0.1
lamda = 0
epochs = 3000

bias = []
variance = []
error = []

x1 = data['x1'].values
x2 = data['x2'].values
y = data['accepted'].values

for degree in range(1, 20):
    X = map_feature(x1, x2, degree)
    theta = np.zeros(X.shape[1])
    # 划分数据集为训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

    # 执行梯度下降函数
    theta, cost = gradientDescent(X_train, y_train, theta,
alpha, epochs, lamda)

    # 计算训练偏差（基于训练集）
    train_probs = predict_probabilities(theta, X_train)
    train_bias = np.mean((train_probs - y_train) ** 2) # 使用
均方误差作为偏差的简化计算

```

```

# 计算测试方差（基于测试集）
test_probs = predict_probabilities(theta, X)
test_variance = np.var(test_probs)

# 计算测试错误率
test_pred_y = [1 if x >= 0.5 else 0 for x in test_probs]
test_error = np.mean(test_pred_y != y)

bias.append(train_bias)
variance.append(test_variance)
error.append(test_error)

plt.plot(range(1, 20), bias, 'r', label='bias')
plt.plot(range(1, 20), variance, 'g', label='variance')
plt.plot(range(1, 20), error, 'b', label='error')
plt.xlabel('degree')
plt.ylabel('Error')
plt.legend()
plt.show()

```

5.3 正则化与偏差方差

当代价函数引入正则化项时，正则化参数 λ 的选择会影响偏差和方差的大小，通过调整参数 λ ，观察偏差和方差的变化，从而选择更好的 λ 。在选择参数 λ 时，为 λ 设置不同的值并进行训练，再用训练好的模型在验证集上进行验证。

Part5-2 动手练

绘制并观察不同正则化参数的偏差与方差，模型定义、代价函数、梯度下降法等定义都与 Part5-1 相同，下面代码直接设置不同的正则化参数并进行训练。

```

# 初始化参数
alpha = 0.1
epochs = 3000
degree = 5

```

```

x1 = data['x1'].values
x2 = data['x2'].values
y = data['accepted'].values
X = map_feature(x1, x2, degree)
theta = np.zeros(X.shape[1])
# 划分数据集为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

bias = []
variance = []
error = []
for lamda in np.arange(0, 0.5, 0.01):
    # 执行梯度下降函数
    theta, cost = gradientDescent(X_train, y_train, theta,
alpha, epochs, lamda)

    # 计算训练偏差（基于训练集）
    train_probs = predict_probabilities(theta, X_train)
    train_bias = np.mean((train_probs - y_train) ** 2) # 使用
均方误差作为偏差的简化计算

    # 计算测试方差（基于测试集）
    test_probs = predict_probabilities(theta, X)
    test_variance = np.var(test_probs)

    # 计算测试错误率
    test_pred_y = [1 if x >= 0.5 else 0 for x in test_probs]
    test_error = np.mean(test_pred_y != y)

    bias.append(train_bias)
    variance.append(test_variance)
    error.append(test_error)

plt.plot(np.arange(0, 0.5, 0.01), bias, 'r', label='bias')
plt.plot(np.arange(0, 0.5, 0.01), variance, 'g',
label='variance')
plt.plot(np.arange(0, 0.5, 0.01), error, 'b', label='error')
plt.xlabel('lambda')
plt.ylabel('Error')
plt.legend()
plt.show()

```

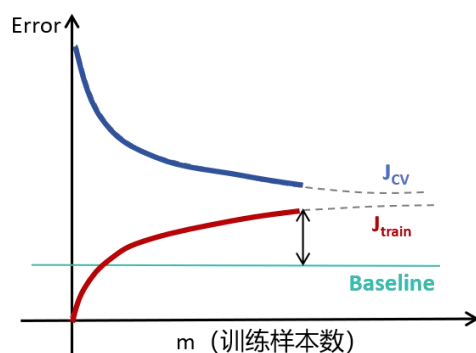
5.4 基准性能水平

在判断一个模型是高偏差还是高方差时，可以设置一个基准性能水平（baseline level of performance）。例如，在语音识别的例子中，任务的目标是将一段语音转成文字，通常一些语音中包含噪音，导致人类也无法准确识别语音中的文字，因此，可以将人类识别语音的误差设置为模型的基准性能水平，而不是要求模型的误差为 0。假设人类识别语音的误差为 5%，则设置 5% 为基准性能水平，当模型在训练集上的误差为 10% 时，则该模型在训练集上的偏差为 $10\% - 5\% = 5\%$ 。当然也可以将基准性能水平设置为 0，此时认为人类有能力做到 100% 的准确率。

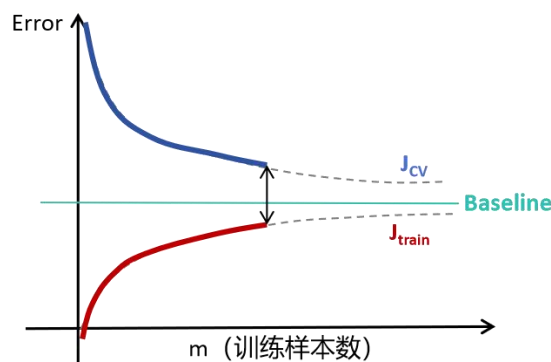
5.5 学习曲线（Learning Curve）

在实际应用中，通常不会直接计算方差，而是通过观察学习曲线判断方差。学习曲线绘制了训练集和验证集（或测试集）上的性能指标（如准确率、误差）与训练数据样本数量之间的关系。学习曲线横轴代表训练数据的样本数量，纵轴代表模型的性能指标。下面我们用误差作为性能指标绘制学习曲线，用 J_{cv} 表示模型在验证集上的误差，用 J_{train} 表示模型在训练集上的误差。

如下图所示为模型具有高偏差时学习曲线可能的形式。



如下图所示为模型具有高方差时学习曲线可能的形式。



回到开头的问题。

Part5-3 动手练

绘制并观察学习曲线，模型定义、代价函数、梯度下降法等定义都与 Part5-1 相同，下面代码直接按不同比例划分训练集与测试集并进行训练。

```
# 初始化参数
alpha = 0.1
epochs = 3000
lamda = 0
degree = 5

x1 = data['x1'].values
x2 = data['x2'].values
y = data['accepted'].values
X = map_feature(x1, x2, degree)
theta = np.zeros(X.shape[1])

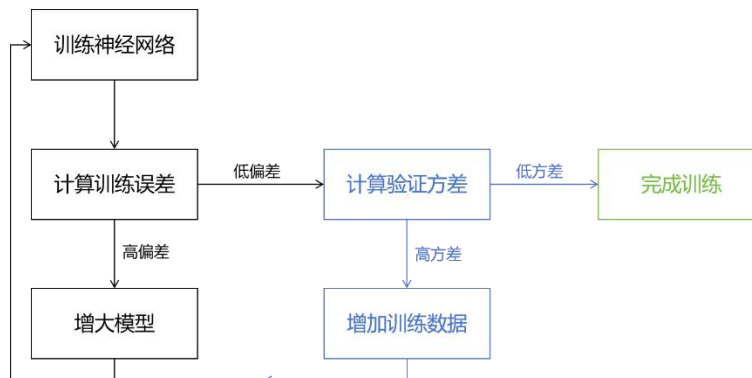
bias_train = []
bias_test = []
for train_size in np.arange(0.1, 0.8, 0.1):
    # 划分数据集为训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(X, y,
train_size=train_size, random_state=42, stratify=y)
    # 执行梯度下降函数
    theta, cost = gradientDescent(X_train, y_train, theta,
alpha, epochs, lamda)

    # 计算训练偏差（基于训练集）
    train_probs = predict_probabilities(theta, X_train)
    train_bias = np.mean((train_probs - y_train) ** 2) # 使用
均方误差作为偏差的简化计算

    # 计算测试偏差（基于测试集）
    test_probs = predict_probabilities(theta, X)
    test_bias = np.mean((test_probs - y) ** 2)
    bias_train.append(train_bias)
    bias_test.append(test_bias)
plt.plot(np.arange(0.1, 0.8, 0.1), bias_train, 'r',
label='train_bias')
plt.plot(np.arange(0.1, 0.8, 0.1), bias_test, 'g',
label='test_bias')
plt.xlabel('m')
plt.ylabel('Error')
plt.legend()
plt.show()
```


5.6 神经网络中的偏差与方差

在前面的介绍过程中，评估模型时需要在偏差与方差之间进行权衡。而在神经网络中，有一种可以无需权衡偏差与方差的方法。事实证明，大型神经网络在小型或中等规模的数据集上训练时，总是具有低偏差。也就是说，当神经网络规模足够大时，几乎总是能很好地拟合数据。因此无需在偏差和方差之间进行权衡，而是根据需要进行降低偏差和方差。



这种方法也存在局限性，因为训练更大的神经网络需要更多的计算资源，当神经网络大过一定程度时，模型过于庞大，训练时间过长，这种方法变得不可行。另一个限制是数据方面的，当训练数据集大到一定程度时，就无法再获取更多的数据。但该方法依然是一种在训练神经网络过程中指导我们选择下一步操作的有效方法。

Part5-4 动手练

计算训练集和测试集误差，模型、训练过程定义与 Part4-4 中使用 PyTorch 的代码相同，下面只给出代码的不同部分。调整隐藏层或者神经元的数量观察模型在训练集的误差，如果误差太大无法接受，可以增加训练轮次，增加层数或神经元数量，如果误差可以接受，则观察模型在训练集的误差和模型在测试集的误差之间的差距（即方差），如果差距太大则增加训练数据继续训练，直至达到比较好的性能。

```
# 测试模型
def predict(model, test_loader):
    model.eval() # 设置为评估模式
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device),
labels.to(device) # 将数据移动到 GPU
            outputs = model.forward(images.reshape(-1, 28 * 28))
```

```

        probabilities =
torch.nn.functional.softmax(outputs, dim=1)
        _, predicted = torch.max(probabilities, dim=1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
    accuracy = 100 * correct / total
    return accuracy

# 加载和预处理数据
train_dataset = datasets.MNIST(root='./', train=True,
transform=transforms.ToTensor(), download=False)
test_dataset = datasets.MNIST(root='./', train=False,
transform=transforms.ToTensor(), download=False)
train_loader =
torch.utils.data.DataLoader(dataset=train_dataset,
batch_size=batch_size, shuffle=True)
test_loader =
torch.utils.data.DataLoader(dataset=test_dataset,
batch_size=batch_size, shuffle=False)

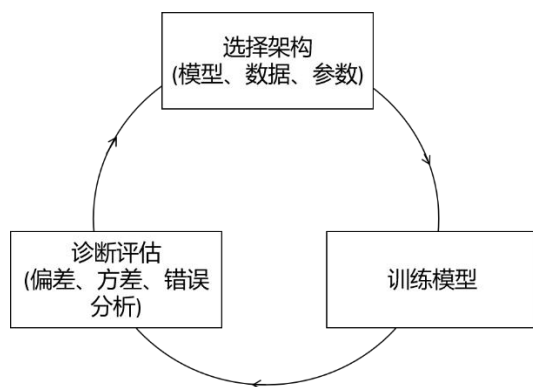
model = NeuralNet(input_size, hidden_size, num_classes)
# 定义损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
# 训练并测试模型
train(model, train_loader, optimizer, criterion, num_epochs=1)
test_accuracy = predict(model, test_loader)
train_accuracy = predict(model, train_loader)

print(f'Validation Error (1 - Accuracy): {100 -
test_accuracy:.2f}%')
print(f'Training Error (1 - Accuracy): {100 -
train_accuracy:.2f}%')

```

5.7 构建机器学习系统的流程

下图为构建一个机器学习系统的总体流程，首先根据任务需求选择模型的类型、训练数据、设置参数，然后对模型进行训练，接下来对训练好的模型进行诊断，评估偏差、方差以及进行错误分析，根据评估和分析结果调整模型、数据和参数再接着训练。在实际应用中，通常需要多次重复此过程直到模型达到理想性能。



5.8 错误分析

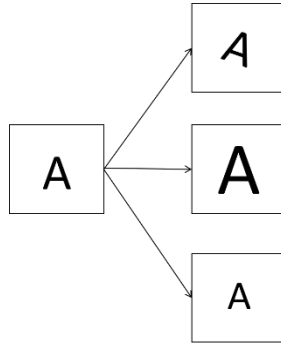
偏差和方差是评估模型性能的两个最重要的指标，错误分析是另外一个决定下一步该怎么做的重要方法。例如，在垃圾邮件分类任务中，我们发现训练好的模型方差高，下一步可以增加训练数据来降低方差，然而在此之前我们可以通过错误分析来确定增加什么样的数据对提升模型性能最有效。下面举例说明错误分析的过程，首先取出 100 个被错误分类的数据，即这 100 条数据的正确标签是垃圾邮件，但模型却没有将它们识别出来，认为它们是正常邮件。人工对这 100 条数据进行查看，发现有 31 条数据是关于股票的，有 3 条是故意拼写错误的，有 7 条是非正常的发件地址，有 8 条是关于钓鱼网站的，有 5 条是将垃圾信息嵌入在图片里的，等等。此时在增加训练数据时，可以有针对性地增加内容中包含股票的数据，而不是没有目的地增加数据。这种方法的目的是把时间放在更重要的事情上，快速提高模型性能。

在机器学习训练过程中，更多的数据总是好的，但获取所有类型的数据可能代价昂贵而且浪费时间，因此可以通过错误分析添加对模型性能影响更大的数据。

5.9 增加训练数据

增加数据除了获取全新的样本之外，还有其他的方法。一种称为数据增强（Data Augmentation）的方法广泛用于图像和音频数据中，数据增强利用现有数据去构建新数据。

在光学字符识别问题中，识别的不仅是数字还包括字母。如下图所示可以对一张图片进行旋转、放大、缩小等处理来获取更多的图片。这些操作不会改变图片的本质，标签不变。



对于语音数据，可以向音频中加入一些人群、汽车等噪声。

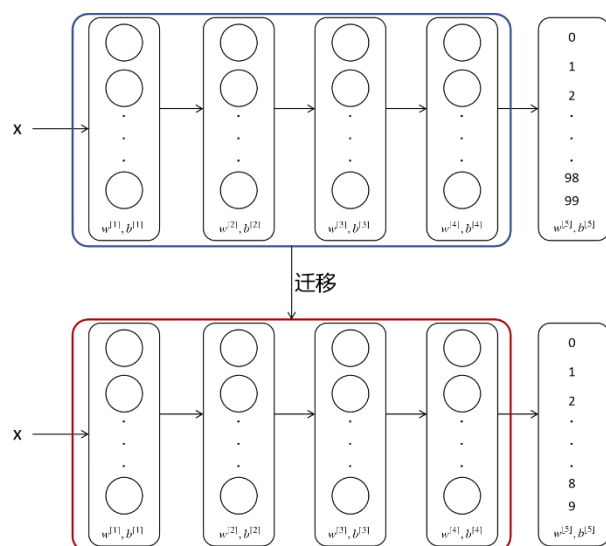
数据增强是一种通过变换和处理现有数据来生成新的训练样本的技术。其核心目的是增加数据的多样性和数量，从而提高模型的泛化能力和鲁棒性。

与数据增强不同的另外一种增加数据的方法称为数据合成 (Data Synthesis)。数据合成是从头开始创建全新的数据，而不是对现有数据进行修改。在光学字符识别问题中，可以用文本编辑器编辑不同字体、大小的字符，然后通过截图获取数据。当然也可以编写代码进行数据合成，这种方式虽然比较耗时，但有时可以极大提升模型性能，因此也是值得的。数据合成通常用于计算机视觉任务，较少用于其他任务。

5.10 迁移学习 (Transfer Learning)

在训练神经网络过程中，当训练数据很少时，迁移学习是一种有效的技术，它允许我们使用来自不用任务的数据来帮助我们训练模型。下面举例说明迁移学习的过程。

假设任务的目标是识别手写数字，但是手写数字样本很少，这时可以找一个数据量充足的其他图像分类数据集训练模型，假设这个数据集有 100 个标签，分别对应猫、狗、羊... 等。如下图上半部分所示，先用这个数据集训练一个五层神经网络，训练完成后将模型的前四层参数迁移到下面的手写数据模型中，由于任务目标不同，最后一层输出层需要需改为 10 个神经元，并初始化 $w^{[5]}, b^{[5]}$ 。接下来可以用手写数字数据集继续训练这个新的模型。训练方式有两种，当训练数据很少时，可以固定前四层参数，只更新输出层参数。当训练数据充足时，可以对全部参数进行更新，此时前四层参数的初始值就是比较好的，从一个好的起点开始训练更容易有希望达到一个好的目标。



为什么迁移学习有效？为什么通过识别一些动物图片学习到的参数能用于识别手写数字这样不同的任务？因为在图像识别问题中，神经网络的第一层可能识别图像的边缘，下一层可能识别简单的角点形状，再下一层可能识别曲线之类的形状。所以神经网络在图像检测问题中，每一层学习的是识别边缘、角点等基本形状，这些在同类型的任务中也是有用的。因此，迁移学习在同类型的任务中有效，但无法应用到不同类型的任务中。

5.11 精确率与召回率

当处理数据集中类别比例不均衡的问题时，偏差和方差通常不再适用。假设任务的目标是根据患者信息预测他患有糖尿病还是一种罕见病，这种罕见病非常少见，在数据集中可能只有 1% 的样本标记为这种罕见病。在这种情况下，即使模型完全没有预测出这种罕见病，误差也只有 1%，偏差和方差都很低，但这个模型效果其实是不好的，因为模型无法预测出罕见病。因此需要其他的评估指标来评估模型性能，下面介绍**精确率**（Precision）和**召回率**（Recall）。

在二分类问题中，测试样本中正样本被判定为正样本的数量记为 TP（True Positive），正样本被判定为负样本的数量记为 FN（False Negative）；负样本被分类器判定为负样本的数量记为 TN（True Negative），负样本被判定为正样本的数量记为 FP（False Positive）。

精确率也称为精度，是指被模型判定为正样本的样本中真正的正样本所占的比例，公式为：

$$P = \frac{TP}{TP + FP}$$

召回率是指所有正样本中被分类器判定为正样本的比例，公式为：

$$R = \frac{TP}{TP + FN}$$

精确率 P 和召回率 R 是一对相互矛盾的量，当 P 高时， R 往往相对较低，当 R 高时， P 往往相对较低，因此很难在二者之间进行权衡。为了更好的评价

模型的性能，一般使用 F1-Score 作为评价标准来衡量分类器的综合性能。

F1-Score: 精确率与召回率的调和平均，F1-Score 更关注的是精确率与召回率较低的那个值。 即：

$$F1 = \frac{1}{\frac{1}{2}(\frac{1}{P} + \frac{1}{R})} = \frac{2PR}{P + R}$$

Part5-5 动手练

计算精确率、召回率、F1-Score。模型、训练过程定义与 Part4-4 中的代码相同，下面只给出代码的不同部分。

```
from sklearn.metrics import precision_score, recall_score, f1_score
import numpy as np

# 测试模型
def predict(model, test_loader):
    model.eval() # 设置为评估模式
    all_labels = []
    all_predictions = []
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device) # 将数据移动到 GPU
            outputs = model.forward(images.reshape(-1, 28 * 28))
            probabilities = torch.nn.functional.softmax(outputs, dim=1)
            _, predicted = torch.max(probabilities, dim=1)
            all_labels.extend(labels.cpu().numpy())
            all_predictions.extend(predicted.cpu().numpy())
    # 计算准确率
    accuracy = 100 * np.mean(np.array(all_predictions) == np.array(all_labels))
    # 计算精确率和召回率
    precision = precision_score(all_labels, all_predictions, average='macro')
    recall = recall_score(all_labels, all_predictions, average='macro')
    f1 = f1_score(all_labels, all_predictions, average='macro')
    return accuracy, precision, recall, f1

# 加载和预处理数据
train_dataset = datasets.MNIST(root='./', train=True, transform=transforms.ToTensor(), download=False)
test_dataset = datasets.MNIST(root='./', train=False,
```

```
transform=transforms.ToTensor(), download=False)
train_loader =
torch.utils.data.DataLoader(dataset=train_dataset,
batch_size=batch_size, shuffle=True)
test_loader =
torch.utils.data.DataLoader(dataset=test_dataset,
batch_size=batch_size, shuffle=False)

model = NeuralNet(input_size, hidden_size, num_classes)
# 定义损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
# 训练并测试模型
train(model, train_loader, optimizer, criterion, num_epochs=1)
accuracy, precision, recall, f1 = predict(model, test_loader)

print(f'Validation Accuracy: {accuracy:.2f}%')
print(f'Precision: {precision:.4f}%')
print(f'Recall: {recall:.4f}%')
print(f'F1-Score: {f1:.4f}%')
```